

Tutorium 10

Laufzeitkomplexität & $\mathcal{O}$ -Notation

Grundlagen der Praktischen Informatik

15. Juni 2026

Wie messen wir Performance?

Praktische Messung

Messung der tatsächlichen Zeit (Benchmarks).

- + Einfach umzusetzen
- Hardware- & Umgebungsabhängig
- Schwer vergleichbar

Theoretische Analyse

Verhalten im asymptotischen Verlauf. Elementarschritte (Zuweisungen, Vergleiche, Add/Sub) zählen als konstante Zeit $\mathcal{O}(1)$.

- + Hardwareunabhängig
- + Fokus auf Skalierbarkeit

Wichtige Erkenntnisse für große n

1. Konstante Faktoren sind egal:

Der Unterschied zwischen $2n$ und $10n$ ist im Vergleich zu n^2 vernachlässigbar.

2. Kleinere Terme verschwinden:

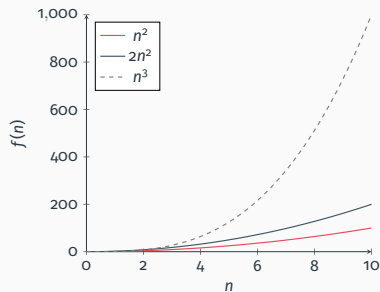
In $n^2 + n + 5$ dominiert n^2 bei großen n vollständig.

Rechenregeln:

- ▶ $\mathcal{O}(c \cdot f(n)) = \mathcal{O}(f(n))$
- ▶ $\mathcal{O}(f(n) + g(n)) = \mathcal{O}(\max(f(n), g(n)))$
- ▶ $\mathcal{O}(f(n)) \cdot \mathcal{O}(g(n)) = \mathcal{O}(f(n) \cdot g(n))$

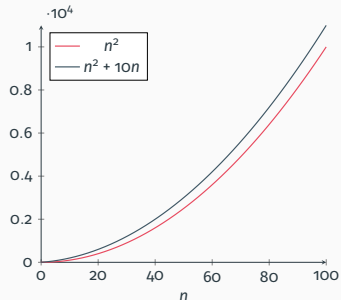
Asymptotisches Verhalten (Visuell)

Konstante Faktoren sind egal



[In Desmos ansehen](#)

Kleine Terme verschwinden



[In Desmos ansehen](#)

Typische Laufzeitklassen, geordnet nach Wachstum (am schnellsten zu am langsamsten):

Klasse	Notation
Konstant	$\mathcal{O}(1)$
Logarithmisch	$\mathcal{O}(\log n)$
Linear	$\mathcal{O}(n)$
Log-Linear	$\mathcal{O}(n \log n)$
Quadratisch	$\mathcal{O}(n^2)$
Kubisch	$\mathcal{O}(n^3)$
Exponentiell	$\mathcal{O}(2^n)$

Live Demo



Altklausuraufgabe 2024_1 (Aufgabe 6)

Wir analysieren zusammen die Laufzeitkomplexität am konkreten Beispiel!

[Altklausur 2024_1 auf Moodle öffnen](#)